

Universidad del Valle de Guatemala

Facultad de Ingeniería
CC3067 Redes — Semestre 2, 2026

Laboratorio 3

Algoritmos de Enrutamiento

Implementación y análisis de algoritmos de enrutamiento sobre una red simulada de
nodos independientes

Integrantes:

Daniel Estrada – 20853
Melisa Mendizabal – 23778
Renato Rojas – 23813
Andre Pivaral – 23574

Guatemala, 6 de septiembre de 2026

1. Descripción de la práctica

Conocer hacia dónde enviar un mensaje vuelve trivial el reenvío: basta consultar el destino final en la tabla de enrutamiento y entregar el paquete al vecino que ofrece la mejor ruta. El problema real no es enrutar, sino *construir y mantener* esa tabla en una red que cambia: enlaces que caen, nodos que se incorporan y nodos que regresan tras una falla. Los algoritmos encargados de esa tarea son los algoritmos de enrutamiento, y son el objeto de esta práctica.

El laboratorio consiste en implementar cuatro de esos algoritmos —Dijkstra, Flooding, Link State Routing y Distance Vector Routing— y probarlos sobre una red simulada. Cada nodo de la red es un proceso independiente que, al arrancar, conoce únicamente la identidad de sus vecinos inmediatos. Toda su visión del resto de la red debe construirla intercambiando mensajes, tal como lo hace un router real, que ve los enlaces conectados a él pero desconoce la topología global.

La práctica se divide en dos fases. La primera utiliza sockets TCP sobre la máquina local como medio de comunicación, lo que permite concentrarse en los algoritmos sin depender de infraestructura externa. La segunda escala esa misma implementación a un servidor XMPP, donde cada nodo corresponde a un usuario de la red y que constituye la entrega oficial. La lógica de enrutamiento es idéntica en ambas fases; lo único que cambia es el medio.

1.1 Objetivos

- Conocer los algoritmos de enrutamiento utilizados en las implementaciones actuales de Internet.
- Comprender cómo funcionan las tablas de enrutamiento.
- Implementar los algoritmos y probarlos en una red simulada sobre un protocolo de capa superior.
- Analizar el funcionamiento y las limitaciones de cada algoritmo.

1.2 Distribución del trabajo

El enunciado establece la implementación de un algoritmo por integrante. La asignación fue la siguiente:

#	Algoritmo	Archivo	Integrante
1	Dijkstra	routing/dijkstra.py	Daniel Estrada
2	Flooding	routing/flooding.py	(por asignar)
3	Link State Routing	routing/lsr.py	(por asignar)
4	Distance Vector Routing	routing/dvr.py	(por asignar)

2. Diseño general de la implementación

Los cuatro algoritmos comparten la misma infraestructura: el formato de los paquetes, el descubrimiento de vecinos, el proceso de reenvío y el medio de transporte. Solo difieren en cómo deciden el siguiente salto. El diseño aprovecha esa separación: existe un único tipo de nodo con dos ejes intercambiables, el algoritmo de enrutamiento y el medio de transporte.

node.py	punto de entrada: CLI, seleccion de algoritmo y transporte
core/	
packet.py	formato JSON del protocolo
config.py	carga de topo-*.txt y names-*.txt, con la restriccion de acces
o	
neighbors.py	descubrimiento de vecinos y costo de enlace via HELLO/ECHO
node.py	proceso de FORWARDING
console.py	consola interactiva del nodo
routing/	LOS ALGORITMOS: un archivo por algoritmo
dijkstra.py flooding.py lsr.py dvr.py	
common/	infraestructura compartida
base.py	interfaz comun que todos implementan
shortest_path.py	Dijkstra puro sobre un grafo (lo usan dijkstra y lsr)
seen_cache.py	deduplicacion de paquetes (la usan flooding y lsr)
transport/	
base.py sockets.py (fase 1) xmpp.py (fase 2)	
configs/	archivos de topologia y de nombres
scripts/	pruebas automatizadas

Figura 1. Estructura del proyecto.

El enunciado exige además que Dijkstra y Flooding sean reutilizables desde Link State Routing sin dejar de funcionar como algoritmos de red autónomos. Por eso el núcleo de Dijkstra vive en `routing/common/shortest_path.py`, sin estado y sin conocimiento de la red: recibe un grafo y un origen, y devuelve distancias y primer salto. Lo mismo aplica a la deduplicación de paquetes inundados, que Flooding y la difusión de LSPs de LSR comparten a través de `common/seen_cache.py`.

2.1 Modelo de nodo: forwarding y routing en paralelo

Cada nodo ejecuta dos servicios simultáneos, según exige el enunciado. El **forwarding** es reactivo: recibe un paquete, determina si es para el propio nodo —en cuyo caso lo entrega al usuario— o consulta la tabla y lo reenvía. Nunca calcula rutas. El **routing** trabaja en segundo plano: emite anuncios periódicos, procesa los de los demás y recalcula la tabla. Nunca manipula los mensajes de usuario. El forwarding *lee* la tabla y el routing la *escribe*; ese es todo el contrato entre ambos.

La concurrencia se resuelve con `asyncio`. Sobre un único bucle de eventos conviven, como tareas independientes, el servidor del transporte, el sondeo periódico de vecinos, las tareas del algoritmo y la consola interactiva. Esto evita las condiciones de carrera propias de los hilos sin renunciar al paralelismo lógico que pide la práctica.

2.2 Protocolo de mensajes

Se respeta el formato JSON definido en el enunciado, común a todos los grupos para permitir la interoperabilidad:

```
{
  "proto" : "dijkstra|flooding|lsr|dvr",
  "type" : "message|echo|info|hello",
  "from" : "usuario_a@alumchat.lol",
  "to" : "usuario_g@alumchat.lol",
  "ttl" : 16,
  "headers": [{"mid":"a1b2c3"}, {"via":"usuario_b@alumchat.lol"}, {"hops":2}],
  "payload": "contenido segun el tipo de paquete"
}
```

Se emplean los cuatro tipos de paquete sugeridos. Los paquetes hello y echo sirven al descubrimiento de vecinos y a la medición de distancias; info transporta la información de enrutamiento (vectores de distancia o LSPs) y message lleva los datos de usuario.

Sobre el campo libre `headers` se definieron cabeceras propias. Todas son opcionales: si un nodo de otro grupo no las envía, la implementación continúa operando y solo pierde la optimización asociada.

Cabecera	Propósito
mid	Identificador único del paquete, base de la deduplicación en Flooding
via	Vecino que realizó el último salto; habilita el split horizon en DVR
t0	Marca de tiempo del HELLO, empleada para calcular el RTT
hops	Salto recorridos por el paquete, con fines de diagnóstico
path	Traza de nodos recorridos, usada para verificar la ruta en las pruebas

2.3 Descubrimiento de vecinos y costo de enlace

Un nodo conoce por configuración la lista de sus vecinos, pero no si están operativos ni a qué distancia se encuentran. Para resolverlo, cada nodo envía periódicamente un paquete HELLO a cada vecino con una marca de tiempo; el vecino responde con un ECHO que devuelve esa marca, y el emisor obtiene así el tiempo de ida y vuelta. El costo del enlace es ese RTT suavizado con un promedio móvil exponencial, lo que amortigua las variaciones puntuales de la máquina.

Este mecanismo es además el detector de fallas del sistema: un vecino que deja de responder durante un intervalo determinado se marca como caído y se notifica al algoritmo de enrutamiento, que es lo que dispara la reconvergencia. Opcionalmente, si el archivo de topología declara pesos explícitos, se utilizan esos valores en lugar del RTT, lo que hace las pruebas deterministas y reproducibles.

2.4 Restricción sobre los archivos de configuración

El enunciado prohíbe emplear los archivos de configuración para algo distinto de configurar el propio nodo y descubrir sus vecinos; en particular, prohíbe leer la topología completa y resolver las rutas de forma estática. La restricción se implementó en el código y no solo en la documentación: el método `NetworkConfig.full_topology()` lanza la excepción `TopologyAccessError` salvo que el nodo se haya construido explícitamente con permiso, cosa que ocurre únicamente cuando el algoritmo seleccionado es Dijkstra puro, el único al que el enunciado concede la topología como entrada.

Los algoritmos dinámicos solo pueden invocar `my_neighbors()`, que devuelve exclusivamente la lista de vecinos propios. De este modo la restricción deja de depender de la disciplina del programador y se convierte en una garantía verificable.

3. Descripción de los algoritmos y su implementación

Los cuatro algoritmos se distinguen por la información de la que disponen, y esa diferencia determina por completo su comportamiento:

Algoritmo	Información de entrada	Naturaleza
Dijkstra	Topología completa	Estático
Flooding	Únicamente sus vecinos	Dinámico, sin tabla
Link State Routing	LSPs de todos los nodos	Dinámico, visión global
Distance Vector	Tablas de sus vecinos	Dinámico, visión local

3.1 Dijkstra

Dijkstra resuelve el problema de caminos mínimos desde un origen en un grafo con pesos no negativos. Parte de una distancia nula al origen e infinita al resto, y de forma iterativa extrae el nodo no visitado de menor distancia conocida y relaja sus aristas: si llegar a un vecino a través del nodo actual resulta más barato que la mejor distancia registrada hasta ese vecino, la distancia se actualiza. El invariante que sostiene su corrección es que, al extraer un nodo, su distancia ya es definitiva, lo cual solo se cumple si ningún peso es negativo.

La implementación emplea un heap binario con borrado perezoso: en lugar de actualizar la prioridad de una entrada existente se inserta una nueva y las obsoletas se descartan al extraerlas, comprobando si el nodo ya fue visitado. El costo resultante es $O((V + E) \log V)$, donde V es el número de nodos y E el de enlaces.

Una tabla de enrutamiento, sin embargo, no necesita la ruta completa hacia cada destino, sino únicamente el *primer salto*. En lugar de reconstruir cada camino al final, el primer salto se propaga durante la relajación: los vecinos directos del origen son su propio primer salto, y cualquier otro nodo hereda el primer salto de su predecesor. Así la tabla queda construida al terminar el recorrido, sin recorridos adicionales.

```

def shortest_paths(graph, source):
    dist = {source: 0.0}; first_hop = {}; previous = {}
    visited = set(); heap = [(0.0, source)]

    while heap:
        d, node = heapq.heappop(heap)
        if node in visited:          # entrada obsoleta (borrado perezoso)
            continue
        visited.add(node)

        for neighbor, weight in (graph.get(node) or {}).items():
            if neighbor in visited or weight is None or weight < 0:
                continue
            candidate = d + weight
            if candidate < dist.get(neighbor, INF):
                dist[neighbor] = candidate
                previous[neighbor] = node
                # el primer salto se hereda del predecesor
                first_hop[neighbor] = neighbor if node == source else first_hop[node]
                heapq.heappush(heap, (candidate, neighbor))

    dist.pop(source, None)
    return dist, first_hop, previous

```

Figura 2. Núcleo de Dijkstra (routing/common/shortest_path.py).

El módulo se mantiene deliberadamente libre de red y de estado. Esa decisión es la que permite que Link State Routing lo reutilice sin modificaciones para calcular sus rutas sobre la topología que reconstruye a partir de los LSPs, que es exactamente lo que solicita el enunciado.

Sobre ese núcleo, routing/dijkstra.py implementa el modo de red: obtiene la topología de la configuración, calcula la tabla al arrancar y responde a las consultas del proceso de forwarding. Puesto que no intercambia información con los demás nodos, es un algoritmo estático y no puede enterarse de los cambios remotos de la red. Lo único que un nodo puede observar por sí mismo es el estado de sus propios enlaces, mediante HELLO/ECHO; ante la caída de un vecino directo la implementación recalcula excluyendo ese enlace, que es todo lo que el algoritmo permite hacer sin violar sus premisas.

3.2 Flooding

[SECCIÓN PENDIENTE — a completar por el integrante responsable de Flooding. Al terminarla, eliminar este recuadro.]

Contenido esperado en esta sección:

- Descripción del algoritmo: el nodo reenvía todo paquete que no es para él por todos sus enlaces activos salvo aquel por el que llegó; no construye tabla.
- Mecanismo de deduplicación por identificador de paquete y por qué es indispensable: sin él, en una topología con ciclos el número de copias crece exponencialmente.
- Papel del TTL como límite superior del recorrido y red de seguridad ante el reinicio de un nodo.

- Fragmento de código representativo de `route()` y de `is_duplicate()`.
- Optimizaciones aplicadas, si las hubo (por ejemplo, entrega directa cuando el destino es un vecino inmediato).

3.3 Link State Routing

[SECCIÓN PENDIENTE — a completar por el integrante responsable de Link State Routing. Al terminarla, eliminar este recuadro.]

Contenido esperado en esta sección:

- Descripción en dos etapas: difusión de LSPs por inundación y cálculo local de rutas con Dijkstra sobre la topología reconstruida.
- Estructura del LSP empleada (origen, número de secuencia y estado de los enlaces propios) y estructura de la base de datos de estado de enlace (LSDB).
- Control de la inundación mediante números de secuencia: por qué es lo que hace que la difusión termine en topologías con ciclos.
- Envejecimiento de los LSPs y su efecto: retirar de la topología a los nodos que dejan de emitir.
- Reutilización explícita de Flooding y de `common/shortest_path.py`, según exige el enunciado.
- Tratamiento de los enlaces declarados por un solo extremo durante la convergencia, si se implementó verificación de bidireccionalidad.

3.4 Distance Vector Routing

[SECCIÓN PENDIENTE — a completar por el integrante responsable de Distance Vector Routing. Al terminarla, eliminar este recuadro.]

Contenido esperado en esta sección:

- Descripción del algoritmo como Bellman-Ford distribuido y planteamiento de la ecuación $D(x,y) = \min_v [c(x,v) + D(v,y)]$.
- Formato del vector anunciado y periodicidad de los anuncios.
- Problema de count to infinity y medidas adoptadas: split horizon con poison reverse y límite de saltos.
- Justificación de que el límite sea de saltos y no de costo: el costo de enlace es el RTT en milisegundos, de modo que un umbral sobre el costo declararía inalcanzables rutas válidas.
- Caducidad de los vectores de vecinos que dejan de anunciar.
- Actualizaciones disparadas (triggered updates) y su efecto sobre el tiempo de convergencia.

4. Resultados

4.1 Escenario de prueba

Las pruebas se realizaron sobre la fase 1 (sockets TCP en la máquina local), levantando siete nodos como procesos independientes. Se utilizó una topología con pesos explícitos para obtener resultados deterministas y reproducibles, verificables a mano:

```
enlaces (costo)   A-B 2    A-C 5    B-C 1    B-D 4
                  C-E 3    D-E 2    D-F 6    E-G 4    F-G 1
```

Figura 3. Topología de prueba (configs/topo-weighted.txt).

El procedimiento automatizado levanta los siete nodos, espera la convergencia, envía un mensaje del nodo A al nodo G y verifica tanto la entrega como la ruta seguida, que viaja registrada en la cabecera path.

4.2 Convergencia y rutas obtenidas

La tabla de enrutamiento calculada por el nodo A fue la siguiente. La ruta óptima hacia G es $A \rightarrow B \rightarrow C \rightarrow E \rightarrow G$ con costo 10, y la implementación la reproduce exactamente:

Destino	Siguiente salto	Costo	Ruta completa
B	B	2	$A \rightarrow B$
C	B	3	$A \rightarrow B \rightarrow C$
D	B	6	$A \rightarrow B \rightarrow D$
E	B	6	$A \rightarrow B \rightarrow C \rightarrow E$
G	B	10	$A \rightarrow B \rightarrow C \rightarrow E \rightarrow G$
F	B	11	$A \rightarrow B \rightarrow C \rightarrow E \rightarrow G \rightarrow F$

Tabla 1. Tabla de enrutamiento del nodo A con el algoritmo Dijkstra.

Merece atención el destino F. Existe un camino más corto en número de saltos ($A \rightarrow B \rightarrow D \rightarrow F$, tres saltos) pero más caro ($2+4+6 = 12$), frente al que efectivamente se elige, de cinco saltos y costo 11. El algoritmo optimiza costo acumulado y no cantidad de saltos, y el resultado lo confirma.

El mensaje enviado de A hacia G fue entregado por la ruta A,B,C,E,G, coincidente con la ruta óptima calculada:


```

=== MENSAJE RECIBIDO ===
de      : A
saltos  : 4
ruta    : A,B,C,E,G
texto   : prueba-dijkstra

```

RESULTADO: OK - mensaje entregado

Figura 4. Salida del nodo G al recibir el mensaje.

4.3 Verificación unitaria del algoritmo

Además de la prueba de red, el núcleo de Dijkstra se verificó de forma aislada contra grafos de respuesta conocida, incluyendo los casos borde que suelen romper una implementación. Las seis familias de prueba, con dieciseis verificaciones en total, se ejecutan sin levantar la red y todas resultaron correctas:

Caso de prueba	Resultado
Distancias y primer salto en la topología con pesos	Correcto
Ruta óptima evitando el enlace caro D-F	Correcto
Topología sin el nodo C: el costo A→G sube de 10 a 12	Correcto
Grafo desconectado: los nodos inalcanzables no entran a la tabla	Correcto
Nodo sin vecinos activos: tabla vacía, sin errores	Correcto
Empate de costos: primer salto estable y válido	Correcto
Vecino directo: el primer salto es el propio destino	Correcto

Tabla 2. Verificación unitaria de routing/common/shortest_path.py.

4.4 Resultados de los demás algoritmos

[SECCIÓN PENDIENTE — a completar por el integrante responsable de Flooding, Link State Routing y Distance Vector Routing. Al terminarla, eliminar este recuadro.]

Contenido esperado en esta sección:

- Tabla de enrutamiento obtenida en el nodo A con cada algoritmo, comparada con la de Dijkstra centralizado de la Tabla 1: una implementación correcta de LSR y de DVR debe converger exactamente a los mismos valores.
- Ruta seguida por el mensaje A→G en cada caso y número de saltos.
- Tiempo aproximado de convergencia observado en cada algoritmo.
- Prueba de robustez: ruta antes de la caída de un nodo intermedio, ruta después de la reconvergencia y, si aplica, ruta tras la reincorporación del nodo.

- En Flooding, cantidad de paquetes duplicados descartados, que ilustra el costo en tráfico del algoritmo.

5. Discusión

El resultado más ilustrativo de la práctica no es que los algoritmos funcionen, sino *cómo* llegan al mismo lugar por caminos opuestos. Dijkstra dispone del mapa completo y calcula la respuesta de una sola vez. Link State Routing no lo tiene, pero lo reconstruye: cada nodo publica el estado de sus propios enlaces, la información se inunda por la red y al final todos poseen el mismo mapa, sobre el cual ejecutan el mismo Dijkstra. Distance Vector nunca llega a ver el mapa; sus nodos solo intercambian distancias y aun así, si la implementación es correcta, convergen a las mismas rutas. Que tres estrategias tan distintas produzcan tablas idénticas es la mejor evidencia de corrección disponible, y por eso la Tabla 1 sirve de referencia para validar los otros algoritmos.

La limitación de Dijkstra quedó expuesta con claridad en las pruebas. Al recibir la topología como entrada y no intercambiar información con los demás nodos, un cambio remoto de la red le resulta invisible: si un enlace lejano desaparece, el nodo sigue calculando rutas sobre una topología que ya no existe y los paquetes se pierden en el camino. Solo puede reaccionar a la caída de sus propios vecinos, que es lo único que percibe directamente. Esta limitación no es un defecto de la implementación sino la definición misma de un algoritmo estático, y es exactamente lo que justifica la existencia de LSR y de DVR.

La restricción del enunciado sobre los archivos de configuración resultó ser, en la práctica, la decisión de diseño más importante. Es tentador leer la topología completa y resolver todo de forma estática, porque el resultado inmediato parece correcto; pero hacerlo convierte a los cuatro algoritmos en el mismo algoritmo y elimina toda capacidad de adaptación. Implementar la restricción como una excepción en el código, y no como una nota en la documentación, obliga a que cada algoritmo construya su visión de la red por los medios que le corresponden.

La elección del RTT medido como costo de enlace, en lugar de un peso arbitrario, acerca la simulación a una red real, pero introduce una dificultad que conviene señalar: los costos dejan de ser números pequeños y pasan a ser milisegundos, con valores que varían entre ejecuciones. Cualquier umbral constante heredado de la literatura —el infinito igual a 16 de RIP es el ejemplo típico— deja de tener sentido, porque un enlace perfectamente sano puede costar 20. Es un error silencioso: la tabla queda vacía sin que ninguna excepción lo indique. Por eso la topología con pesos explícitos resultó indispensable para las pruebas: separa los errores del algoritmo del ruido de la medición.

Finalmente, la separación entre el núcleo del algoritmo y su modo de red demostró su valor más allá de la limpieza del código. Al mantener `shortest_path.py` sin estado y sin conocimiento de la red, fue posible verificarlo de forma unitaria contra respuestas calculadas a mano, sin levantar siete procesos ni esperar la convergencia. Depurar un algoritmo distribuido observando únicamente su comportamiento agregado es considerablemente más difícil que verificar por separado la pieza determinista.

6. Conclusiones

- El problema central del enrutamiento no es reenviar paquetes, sino construir y mantener la tabla que hace posible ese reenvío. Los cuatro algoritmos resuelven la misma pregunta con información distinta, y la cantidad de información de la que disponen determina por completo sus capacidades y sus limitaciones.
- Dijkstra produce rutas óptimas de forma inmediata cuando la topología es conocida y estable, pero al no intercambiar información con los demás nodos no puede adaptarse a los cambios remotos de la red. Su valor real en esta práctica es doble: sirve como referencia de corrección para los algoritmos dinámicos y constituye el motor de cálculo de Link State Routing.
- La modularidad exigida por el enunciado no es un requisito estético. Mantener el núcleo de Dijkstra libre de estado y de red permitió reutilizarlo sin modificaciones y verificarlo de forma aislada, lo que redujo notablemente la dificultad de depuración frente a la alternativa de probar todo el sistema distribuido en conjunto.
- Implementar en el código la restricción de acceso a la topología, mediante una excepción en lugar de una advertencia en la documentación, resultó ser una decisión acertada: convierte una regla del enunciado en una garantía verificable y evita el atajo que anularía el propósito del laboratorio.
- Separar la lógica de enrutamiento del medio de transporte permitió desarrollar y probar por completo la primera fase sobre sockets TCP, dejando el paso al servidor XMPP como un cambio de una sola clase, sin tocar los algoritmos.

7. Comentarios

El aspecto que mayor coordinación exige de cara a la prueba en clase no es algorítmico sino de acuerdo entre grupos. El enunciado fija la estructura del paquete, pero deja libre el contenido del campo payload de los paquetes de tipo info, que es justamente donde viajan los vectores de distancia y los LSPs. Si cada grupo define ese contenido por su cuenta, Flooding funcionará entre implementaciones distintas—no necesita acuerdo alguno— pero LSR y DVR se ignorarán mutuamente, descartando como malformada la información del otro. Conviene fijar ese formato antes de la sesión de pruebas.

Como medida preventiva, la implementación trata todas las cabeceras propias como opcionales: si un nodo de otro grupo no las envía, el algoritmo continúa operando y únicamente pierde la optimización asociada, sin romper la interoperabilidad.

8. Referencias

- Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8.^a ed.). Pearson.
- Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5.^a ed.). Prentice Hall.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.

Malkin, G. (1998). *RIP Version 2* (RFC 2453). Internet Engineering Task Force.
<https://www.rfc-editor.org/rfc/rfc2453>

Moy, J. (1998). *OSPF Version 2* (RFC 2328). Internet Engineering Task Force.
<https://www.rfc-editor.org/rfc/rfc2328>

Saint-Andre, P. (2011). *Extensible Messaging and Presence Protocol (XMPP): Core* (RFC 6120). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6120>

Python Software Foundation. (2026). *asyncio — Asynchronous I/O*.
<https://docs.python.org/3/library/asyncio.html>